

PROJECT_A

TECHNICAL SPECIFICATION // CLASSIFIED

GENERATED BY: GEMINI CLI
DATE: 2026-02-05

OVERVIEW

ProjectA is an **Unreal Engine 5** based Multiplayer Action RPG/Shooter. It features a robust architecture integrating modern UE5 features with custom backend services.

KEY SYSTEMS

1. INVENTORY & EQUIPMENT (`ATGINVENTORYCOMPONENT`, `ATGEQUIPMENTCOMPONENT`)



FIG 01.1: SPATIAL INVENTORY MATRIX

- > **Grid-Based System:** Implements a "Tetris-style" inventory using `FInventoryGrid` (derived from `FFastArraySerializer`) for network efficiency.
- > **Placement Logic:** Features algorithms for spatial management (`FindFirstFit`, `CanPlaceRect`) considering item dimensions and rotation.
- > **Equipment:** Manages slots (Main Weapon 1/2, etc.) via `ATGPlayerEquipComponent`, handling mesh attachment and animation states.
- > **Data Driven:** Uses `UATGItemData` as a base for all items, extending to `UATGWeaponData`, `UATGConsumableItemData`, etc.

2. COMBAT SYSTEM

- > **Weapons:**
- > **Ranged (`AATGRangeWeapon`):** Hybrid hit-scan/projectile logic.
- > **Melee (`AATGMeleeWeapon`):** Uses `AnimNotifyStates` for precise hit windows and sub-frame trajectory interpolation for accurate hit detection.
- > **Bullet Manager (`ABulletManager`):** Optimizes performance by managing projectiles as lightweight structs (`FBullet`) in a centralized simulation

loop, avoiding the overhead of spawning individual actors.

- > **Gore/Slicing** (`USliceSystemComponent`): Utilizes `ProceduralMeshComponent` and **GeometryScript** to dynamically slice skeletal meshes based on bone hierarchy, recalculating skin weights for realistic deformation.

3. GAMEPLAY ABILITY SYSTEM (GAS)

- > **Core Integration**: Implemented on `AATGPlayerCharacter` and `AZombieEnemy`.
- > **Attributes**: `UCharacterAttributeSet` manages Health, MaxHealth, and Damage.
- > **Abilities**: `UGA_MeleeCombo` demonstrates combo logic using tasks like `WaitInputPress` and custom target data tasks (`WaitMeleeTargetData`).

4. AI [`ABASEAICONTROLLER`, `AZOMBIEENEMY`]

- > **Perception**: Uses `AIPerception` for sight-based target detection.
- > **Behavior**: Implements Behavior Trees with custom tasks for distance checking and montage playback.
- > **State Machine**: Manages states (Normal, Chase, Battle) via Blackboard keys.

5. NETWORKING & BACKEND

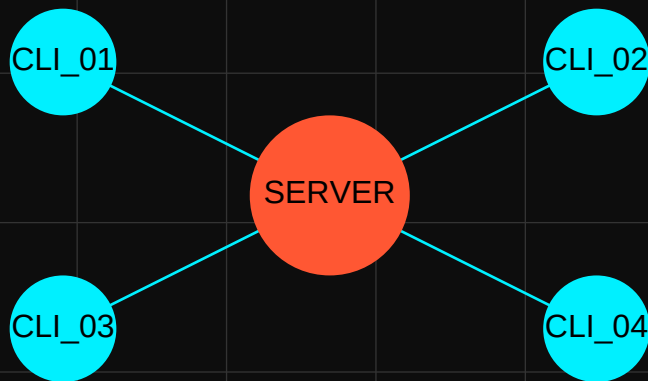


FIG 02.0: CLIENT-SERVER REPLICATION TOPOLOGY

- > **Subsystem** (`UNetworkGameInstanceSubsystem`): Handles HTTP requests for authentication (Login/Register) and persistence (Inventory/Stash save & load).
- > **Real-time Communication**: Features a TCP socket worker for chat systems using **FlatBuffers**.
- > **Session Management**: Includes a Lobby system (`LobbyGameMode`) and Steam session integration.

ARCHITECTURE

> ****Interfaces:**** heavily relies on interfaces (`IATGInterface`,
`IATGInventoryOwnerInterface`) to decouple systems.
> ****Optimization:**** Focuses on network bandwidth and CPU performance
through FastArray serialization and centralized simulation managers.
> ****Modern UE5:**** Leverages Enhanced Input and Geometry Scripting.

